PURDUE UNIVERSITY | Department of Computer Science

# Statement of Purpose

*Shangyin Tan*

My primary research interests are in the field of programming languages, software engineering, and program analysis. My current research focuses on analyzing, designing, and implementing efficient program analysis tools from a functional programming perspective. Program analysis tools are crucial in real-world software development, such as software testing, input generation, verification, and dataflow analysis. The expectation for program analysis tools often comes from two categories: reliability and speed. Therefore, the critical question to develop a practical program analysis tool is how to achieve correctness and efficiency simultaneously.

To address this question, I have been working with Professor Tiark Rompf, focusing on building efficient symbolic execution engines. Symbolic execution is a program analysis technique that automatically explores the program execution space by treating program inputs as symbolic values. Traditional symbolic execution engines are often implemented as interpreters, which are likely to be correct but slow. We propose a semantic-first approach to implement symbolic execution engines as compilers through multi-stage programming, where we try to find the synergy between correctness and efficiency. Our approach starts with the symbolic execution interpreter, which is close to semantics thus clear to argue about correctness. Then we specialize it to the target program via partial evaluation to obtain the compiled symbolic execution program for more efficient analysis.

When I joined the project, the preliminary methodological work was done and implemented for a small imperative language. I was excited about this idea but wanted to apply it to more complex languages. I extended the implementation to target LLVM IR (a general intermediate representation language), supporting more than 40 instructions and additional data types, including arrays, structs, and pointers. Then, I integrated support for SMT solvers, which was missing in the original prototype. Finally, I developed a working symbolic execution compiler for LLVM IR that correctly generates inputs for programs with finite execution paths and outperforms the state-of-the-art symbolic execution engine KLEE on both synthetic and small realistic benchmarks. This work was published in our OOPSLA [4] and ESEC/FSE [3] papers.

As we had been focusing on a specific symbolic execution variant called fork-based symbolic execution, I wanted to see if this approach could be adapted to cover other symbolic execution strategies like concolic execution or backward symbolic execution. From this point of the project, I took the lead from my collaborators. I focused on developing the theory and implementing the tool for concolic execution, which uses concrete values for exploration and symbolic values for input generation. For the theory side, we approached the concolic execution semantics by modularly composing concrete semantics and symbolic semantics. Then, we showed that one could directly obtain the concolic execution interpreter by simply combining the concrete and symbolic interpreters using higher-order functional combinators. Finally, we demonstrated how this concolic interpreter

could be turned into a concolic compiler using similar partial evaluation techniques. We will present this part as a short paper at PEPM [2] (I am the first author).

After I completed the operational semantics of concolic execution, I implemented the concolic compiler targeting LLVM IR. Our prototype concolic compiler achieved similar performance to the state-of-the-art concolic execution compiler SymCC on benchmarks like different sorting programs. I also investigated another symbolic execution variant called backward symbolic execution that performs the symbolic execution backward from a target location. After tackling a few challenges arising from backward execution like code generation for out-of-order execution and runtime support for backward function call traces, I used the same partial evaluation technique to develop the backward symbolic execution compiler. Finally, we distilled a general recipe of compiling the symbolic execution approaches from our previous work, formulating a principled approach to build symbolic execution compilers, that is both language agnostic and strategy agnostic. We submitted this work to USENIX Security [1].

I plan to work towards a PhD degree in programming languages and software engineering. My future research vision includes broader explorations on how to analyze programs and how to build efficient program analysis tools for developers. Specifically, I want to see how to better use compilers for program analysis and how to make better program analyzers for compilers. In addition, I have been working with Professor Benjamin Delaware on a formal verification project. The interaction with verification inspires me to work on *correct* program analysis tools as well. Current program analysis tools often lack rigorous reasoning about correctness. If I can formally prove a program analysis tool is correct, software developers can utilize it without regret. My ultimate goal is to build *efficient* and *correct* program analysis tools to help programmers write better, safer, and more efficient programs.

## References

[1] Shangyin Tan, Guannan Wei, and Tiark Rompf. The essence of compiling symbolic execution. In *USENIX Security Symposium*. USENIX Association, 2022 (in submission)

[2] Shangyin Tan, Guannan Wei, and Tiark Rompf. Towards partially evaluating symbolic interpreters for all (short paper). In *PEPM@POPL*. ACM, 2022 (to appear)

[3] Guannan Wei, Shangyin Tan, Oliver Bracevac, and Tiark Rompf. LLSC: a parallel symbolic execution compiler for LLVM IR. In *ESEC/SIGSOFT FSE*, pages 1495–1499. ACM, 2021

[4] Guannan Wei, Oliver Bracevac, Shangyin Tan, and Tiark Rompf. Compiling symbolic execution with staging and algebraic effects. *Proc. ACM Program. Lang.*, 4(OOPSLA):164:1–164:33, 2020